

SE Lab 4 - Endless Runner Game

Name: Shobhit Rachit Keshri

SRN: PES2UG24CS475

Section: H

Note: I used Claude (Opus 5.5 Medium)

Plugins Used

Ponytail, Caveman, Agent Skills, ECC

The plugins meant I didn't have to repeat the same instructions in every prompt: Ponytail kept changes minimal and Caveman kept replies short, so my prompts could focus only on the task. Agent Skills and ECC made Claude understand the code before editing and review its own work

Task 1: Refine Collision Detection

This is a Pygame endless runner. Files: `main.py`, `game/game_engine.py`, `game/player.py`, `game/obstacle.py`.

Bug: obstacle speed increases forever, and once speed exceeds the obstacle width per frame, obstacles teleport past the player and the collision is never detected (tunnelling).

Before changing anything: read `game_engine.py` and `obstacle.py` and tell me exactly which lines cause the speed to grow unbounded and where the per-frame movement happens. Then propose a fix that does two things:

(1) caps the speed so it can never exceed the smallest obstacle width per frame, and (2) makes collision detection robust to fast movement (e.g. swept/segmented movement instead of a single big jump).

Give me the minimal diff, not a rewrite. Explain why each changed line is needed.

+

↩

Claude is AI and can make mistakes. Please double-check responses.

Opus 5.5 Medium

Task 2: Implement Game Over Condition

Now add a Game Over screen. Requirements:

1) When the player collides, stop the run and draw a screen showing "GAME OVER" and the final score

2) No console print; everything on the Pygame surface

3) The screen must keep handling the event queue so the window stays responsive and can be closed with the X button or Esc

Do not restructure the game loop into new classes. Show me the smallest change to `game_engine.py` that adds a game-state flag and a draw function. Point out any place where the old code would now run when it shouldn't.

+

↩

Claude is AI and can make mistakes. Please double-check responses.

Opus 5.5 Medium

Task 3: Add Replay Option

Extend the Game Over screen with a replay option:

- 1) Keys 1/2/3 choose Easy, Medium, Hard, which set the starting obstacle speed and spawn interval
- 2) Q or Esc exits.
- 3) Choosing a difficulty fully resets score, player position, obstacles and speed — no state leaking from the previous run

List every piece of mutable state in the engine that must be reset, then show the diff. Tell me which of those would have been missed by a naive "just set score = 0" reset.

+

↩

Claude is AI and can make mistakes. Please double-check responses.

Opus 5.5 Medium

Task 4: Add Sound Feedback

Add sound effects: jump, score increment, and game over

Constraints:

- 1) Must not crash if the audio device is unavailable or the sound files are missing — wrap init and loading so the game still runs silently
- 2) No new dependencies beyond pygame
- 3) Generate the three sounds programmatically (pygame.sndarray / numpy-free if possible) or load from a sounds/ folder, whichever is fewer lines — tell me which you picked and why

Show the diff and the exact place each sound is triggered

+

↩

Claude is AI and can make mistakes. Please double-check responses.

Opus 5.5 Medium

Code Review Prompt

Asked Claude to critically review its own changes without defending them.

Review the change you just made as a critical code reviewer. Find: any edge case it breaks, any dead code left behind, and anything simpler that would work. Do not defend the code.

+

↩

Claude is AI and can make mistakes. Please double-check responses.

Opus 5.5 Medium